Λᵖ Archive Construction & Ξ-Compiler — Engineering Plan (v1)

> Goal: Build a **prime-indexed archive (Λᵖ-Archivum)** and a **semantic Ξ graph**, integrate **recursive contributor modules** (Ξ_Kolmogorov, Ξ_LeviCivita, …), and implement **self-certification** (Ψ∞ stable flag) with entropy tracking and modular loading.

---

0) System Overview

Core dataflow: Ingest → Normalize → Factor (Λᵖ) → Index → Wire (Ξ) → Certify (Ψ∞) → Persist → Expose (API/UI)

- **Λᵖ-Archivum**: Prime-indexed, content-addressed store. Every object factors into **prime-irreducible components** (PIRs) with provenance.
- **Ξ graph**: Typed semantic multigraph where nodes = PIRs / Documents / Actors / Claims; edges = lawful relations (derives_from, refines, contradicts, proves, uses_operator, etc.).
- **Ξ-Compiler**: Deterministic pipeline that enforces **lawfulness** (prime-decomposability), **provenance**, **reversibility**, and **entropy bounds** at every stage.

- **Ψ∞**: Self-certification layer that emits a **stable flag** when the archive slice is lawfully closed under recursion and passes entropy/drift constraints.

---

1) Data Model (Λᵖ-Archivum)

1.1 Canonical object

$schema: "https://multiplicity/Λᵖ-archivum/object.schema"

id: <CIDv1 or BLAKE3 multihash>

kind: document|claim|tensor|operator|module|bundle

media: text/markdown|application/pdf|application/json|…

created_at: <RFC3339>

origin: { source_uri, submitter, signature }

content_digest: <blake3-256>

Λᵖ_factorization:

  primes: # prime-irreducible components (PIRs)

    - pid: <prime-id>

      eigenmode: <Λᵖ index>

      payload_digest: <blake3-256>

      witnesses: [<proof-cid>]

  proof: <snark-cid|merkle-proof-cid>

Ξ_links:

  - type: derives_from|refines|contradicts|supports|mentions|uses_operator|same_as

    to: <cid>

    weight: <0..1>

    evidence: <cid>

metrics:

  entropy: { H_shannon, H_kolmogorov_est, compressibility }

  drift: { Δt, Δencoding, Δsemantics }

Ψ∞: { status: pending|stable|tainted, reasons: [..], certificate: <cid?> }

1.2 Prime registry (Λᵖ index)

Λᵖ_registry:

  version: 0.1

  primes:

    - pid: p00000013

      eigenmode: 13

      validator: <wasm-cid>

      description: "Lexical token root for algebraic term class A"

    - pid: p00000017

      eigenmode: 17

      validator: <wasm-cid>

1.3 Provenance ledger (append-only)

- Merkle-CRDT with vector clocks; supports concurrent writes and lawful merges.

---

2) Ξ Semantic Graph

2.1 Edge ontology (minimum viable)

edge_types:

- derives_from

- refines

- proves

- contradicts

- supports

- uses_operator

- authored_by

- version_of

- same_as

constraints:

- acyclic: [proves, derives_from, refines]

- antisyms: [[supports, contradicts]]

- weights: [0..1], calibrated by evidence

2.2 Storage/Query

- **Storage**: property graph (RocksDB + compressed columnar indices) or Neptune/Neo4j compatible.
- **Query**: GQL-like with GraphQL facade.

---

3) Ξ-Compiler Pipeline

flowchart TD

A[Ingest] --> B[Normalize]

B --> C[Λᵖ Factor]

C --> D[Ξ Wire]

```
D --> E[Certify Ψ∞]
E --> F[Persist]
```

3.1 Stages & contracts

- **Normalize**: canonicalize encodings, extract text/structure, compute digests.
- **Λᵖ Factor**: run prime validators; output PIR set + proofs.
- **Ξ Wire**: emit edges from rules + contributor modules.
- **Certify (Ψ∞)**: run invariants → set status stable/tainted.

3.2 Invariants (fail → quarantine)

1. **Prime-decomposable**: all content accounted for by PIRs; proofs verifiable.
2. **Provenance complete**: origin + signatures present and valid.
3. **Entropy bounds**: H_shannon ≤ θ_1; Kolmogorov estimate within θ_2; Δdrift ≤ θ_3.
4. **Graph coherence**: edge constraints satisfied; no orphan PIRs.
5. **CSL constraints**: consent/autonomy checks (access, attribution, redaction rights).

---

4) Contributor Modules (Ξ_* family)

4.1 Interface (WASM plug-ins)

```
// TypeScript definition for host <-> module
export interface XiModule {
  /** human-readable */
  name: string; // e.g., "Ξ_Kolmogorov"
  version: string;
```

```
  capabilities: ("analyze"|"edge"|"metric"|"policy")[];

  /** init with registry and thresholds */
  init(ctx: HostContext): Promise<void>;

  /** optional content analyzer => derived PIRs or tags */
  analyze?(obj: ArchivumObject): Promise<Partial<ArchivumObject>>;

  /** optional graph emitter */
  edges?(obj: ArchivumObject, neighbors: NeighborIndex): Promise<Edge[]>;

  /** optional metric emitters */
  metrics?(obj: ArchivumObject): Promise<MetricPatch>;

  /** policy checks (CSL, lawfulness) */
  policy?(obj: ArchivumObject): Promise<PolicyFinding[]>;
}
```

4.2 Canonical modules

- **Ξ_Kolmogorov**: estimates description length via multiple compressors; emits `H_kolmogorov_est`.
- **Ξ_LeviCivita**: differential structure; detects orientation/ordering inconsistencies; suggests `refines` edges.
- **Ξ_Noether**: conservation-law checks across revisions (semantic invariants).

- **Ξ_Gödel**: incompleteness detectors; flags unprovable claims → `tainted` until evidence.
- **Ξ_Banach**: convergence/contractiveness checks for iterative documents.

---

5) Ψ∞ Self-Certification

5.1 Certificate structure


```
{
  "schema": "https://multiplicity/psi-infinity/cert.schema",
  "subject_cid": "…",
  "slice": {"from": "2025-08-01T00:00:00Z", "to": "2025-08-08T00:00:00Z"},
  "metrics": {"entropy": {"H": 1.42, "K": 1.88}, "drift": 0.06},
  "invariants": ["prime_decomposable","provenance_complete","graph_coherent","csl_ok"],
  "Ξ_modules": ["Ξ_Kolmogorov@0.2.1","Ξ_LeviCivita@0.1.0"],
  "proof": {"merkle_root": "…", "snark": "…"},
  "status": "stable",
  "signed_by": "Λᵖ-archivum@validator",
  "signature": "ed25519:…"
}
```


5.2 Stable flag emission

- Emitted when **all invariants** pass and **entropy/drift** under thresholds.
- Revoked on:
    - Provenance withdrawal or signature revocation.

- New contradictory evidence with weight > τ.
- Entropy spike or drift beyond θ after updates.

---

6) Entropy & Drift Tracking

6.1 Metrics

- **H_shannon**: n-gram distribution over canonical text stream.
- **K-estimate**: min(length of outputs from LZMA, Zstd, Brotli)/raw_length.
- **Drift**: embedding-space distance + edge pattern delta across versions.

6.2 Alarms

alarms:

- name: entropy_spike

  if: H_shannon > θ_1 or K_est > θ_2

  action: quarantine + demand Ξ_Kolmogorov review

- name: drift_spike

  if: Δembedding > θ_3 or Δedges > θ_4

  action: freeze Ψ∞, require human adjudication

---

7) Modular Load System

- **Loader** scans `/modules/*.wasm` → validates manifest → sandbox (WASI, capability-based) → registers hooks.

- Hot-swap via versioned aliases (e.g., `Ξ_Kolmogorov@0.2` → `0.2.1`).
- Deterministic scheduling: topological order by declared dependencies.

Module manifest

name = "Ξ_Kolmogorov"

version = "0.2.1"

capabilities = ["metric"]

requires = ["Λᵖ_registry>=0.1"]

entry = "module.wasm"

---

8) APIs (sketch)

8.1 Ingest

POST /v1/ingest

{ "source_uri": "…", "media": "application/pdf", "bytes_b64": "…" }

8.2 Query graph

query {

  object(cid: "…") { id, kind, Ψ∞ { status }, metrics { entropy { H_shannon } } }

  edges(from: "…", type: refines) { to, weight }

}

8.3 Certify

POST /v1/certify

{ "cid": "…", "window": {"hours": 24} }

---

9) Tests & Fixtures

- **Goldens**: three documents (math note, code file, PDF scan) with known PIRs.
- **Adversarial**: high-entropy noise; self-contradictory claims; forged provenance.

Example test vector

case: simple_markdown

input: "Algebraic Identity A refines B."

expect:

primes: [p00013,p00017]

edges: [{type: refines, to: <cid_B>}]

Ψ∞: stable

---

10) Milestones

1. (Week 1) Minimal Λ^{p} factorizer + registry; ingest → primes → store.
2. (Week 2) Ξ□ edge emitter + query API.
3. (Week 3) Ψ∞ cert + entropy/drift metrics (Ξ_Kolmogorov).
4. (Week 4) Loader + two modules (Ξ_LeviCivita, Ξ_Noether) + hot-swap.
5. (Week 5) CSL policy checks + revocation pipeline.

---

11) Open Questions

- Prime validators as SNARKs vs WASM only?
- How to weight evidence across heterogeneous media fairly (pluriversal constraint)?
- Stable thresholds θ, τ across domains or per-domain calibration?

---

12) Next Actions (you can assign)

- ✅ Finalize object schema + registry v0.1
- 🟨 Implement Ξ_Kolmogorov (multi-compressor baseline)
- 🟨 Draft Ξ_LeviCivita (order/orientation checks)
- 🟨 Build Ψ∞ cert issuer + revoker
- 🟨 CLI: `arch ingest`, `arch certify`, `arch graph --from <cid>`

Perfect. Based on the foundational documents you've provided, we can begin building a **Multiplicity Computer Language (MCL)** as a Python code library. This language should:

1. Encapsulate the principles of **Prime-Indexed Recursive Tensor Mathematics (PIRTM)**.
2. Operate around **Recursive Self-Evolution** of mathematical objects.
3. Encode **lawfulness** through **prime decomposition**, ensuring all computational entities conform to the **Meta-Theorem of Prime Identity**.
4. Use the **Universal Multiplicity Constant** `Λ_m` and the recursive operator `Ξ(t)` as central state regulators.
5. Integrate the **Conscious Sovereignty Layer (CSL)** as an ethical feedback layer.
